

Efficient Pipeline Parallelism for Reinforcement Learning

Faiz Sameer Ahmed
Computer Science
San Jose State University
San Jose, USA
faiz.ahmed@sjsu.edu

Dr. Genya Ishigaki
Computer Science
San Jose State University
San Jose, USA
genya.ishigaki@sjsu.edu

Abstract—Reinforcement learning trains agents through interaction with environments, alternating between a rollout phase where experiences are collected and a training phase where policy and value networks are updated. Traditional distributed RL architectures transfer collected experiences to a centralized server that trains the actor-critic network and returns updated parameters. However, when deploying large neural networks in resource-constrained environments that require model parallelism across multiple devices, communication overhead increases substantially. Pipeline parallelism necessitates transferring both forward activations and backward gradients between devices for each training epoch, which can be repeated 10 or more times per batch for sample efficiency. This paper presents a gradient accumulation technique that selectively transmits high-magnitude gradients above a specified percentile threshold during backward propagation. By accumulating gradients over time and only communicating the most significant values, we reduce network transfer by 35 percent while maintaining comparable training performance to full gradient communication. We further investigate buffer decay as a mechanism to mitigate gradient staleness, finding that per-minibatch decay interacts poorly with PPO’s multi-epoch training and degrades performance.

Index Terms—Reinforcement Learning, Pipeline Parallelism, Gradient Compression, Distributed Training, Model Parallelism

I. INTRODUCTION

Reinforcement learning trains intelligent agents through environmental interaction, alternating between rollout phases where experiences are collected and training phases where policies are updated. As neural network architectures grow increasingly complex, computational and memory requirements often exceed single-device capabilities, necessitating distributed training approaches.

Traditional distributed RL systems use centralized architectures where edge devices collect experiences while cloud servers train models. However, when models become too large for a single device’s memory, pipeline parallelism becomes essential, partitioning networks into sequential stages across multiple devices. Unlike centralized training where only experiences and parameters are transferred, pipeline parallelism requires continuous exchange of forward activations and backward gradients between devices. This communication occurs repeatedly as we run multiple training passes over the data for sample efficiency, creating substantial overhead in bandwidth-limited environments.

This paper addresses this communication challenge through selective gradient transmission. We present a gradient accumulation technique that identifies and communicates only the most significant gradients based on magnitude percentile thresholds. Section II reviews related work. Section III defines the problem and quantifies communication overhead. Section IV describes our method. Section V presents experimental results demonstrating network transfer reduction of 35 percent while maintaining comparable performance, and analyzes the effect of buffer decay on gradient accumulation.

II. RELATED WORK

A. Distributed Reinforcement Learning

Early work on distributed RL established foundational architectures for scaling training. Nair et al. [1] introduced massively parallel methods for deep RL using parallel actors generating behavior, parallel learners trained from stored experience, and distributed neural networks, achieving order-of-magnitude speedups on Atari games. Building on this, Espeholt et al. [2] proposed IMPALA, which decouples acting from learning using importance sampling for off-policy corrections, scaling to thousands of machines while achieving state-of-the-art results on multi-task benchmarks. Horgan et al. [3] introduced Ape-X, separating data collection from learning through distributed prioritized experience replay, where multiple actors generate experience stored in a shared replay buffer accessed by learners.

B. Communication-Efficient Distributed Training

Reducing communication overhead in distributed training has been extensively studied. Chen et al. [4] developed communication-efficient policy gradient methods specifically for distributed RL, proposing gradient compression techniques that reduce communication costs while maintaining convergence guarantees. In the supervised learning domain, Lin et al. [5] discovered that 99.9% of gradient exchange in distributed SGD is redundant, proposing Deep Gradient Compression (DGC) that achieves 270-600x bandwidth reduction through momentum correction and local gradient clipping. Alistarh et al. [6] introduced QSGD, a family of gradient quantization schemes providing convergence guarantees for both convex

and non-convex optimization while allowing smooth trade-offs between communication bandwidth and convergence time.

C. Pipeline Parallelism for Large Models

Pipeline parallelism has emerged as a critical technique for training models exceeding single-device memory. Huang et al. [7] presented GPipe, which partitions neural networks across multiple accelerators and pipelines mini-batches through stages, introducing micro-batching and gradient accumulation to achieve near-linear speedup while training models 25x larger than single-accelerator capacity. Narayanan et al. [8] proposed PipeDream, which automatically partitions DNN models and schedules computation to minimize pipeline bubbles, maintaining multiple parameter versions to enable weight updates without stalling. For transformer models, Shoeybi et al. [9] developed Megatron-LM with efficient intra-layer model parallelism, achieving 76% scaling efficiency on 512 GPUs for 8.3 billion parameter models. Rajbhandari et al. [10] introduced ZeRO, eliminating redundant storage of optimizer states, gradients, and parameters across data-parallel processes, enabling training of 170 billion parameter models.

D. Resource-Constrained Environments

Recent work addresses distributed learning in resource-constrained settings. Lim et al. [11] applied federated learning principles to RL for IoT devices, enabling multiple agents to collaboratively learn optimal control policies while keeping data local, addressing device-specific dynamics variations. Feng et al. [12] designed IoTSL, an efficient split learning system for resource-constrained IoT devices, optimizing the split learning paradigm to handle non-IID data distribution and reduce communication requirements specific to IoT constraints.

Our work differs from prior approaches by specifically addressing the communication bottleneck in pipeline-parallel RL training through selective gradient accumulation. While existing gradient compression techniques focus on compressing all gradients uniformly or through quantization, our approach accumulates gradients over time and transmits only high-magnitude values above configurable percentile thresholds, tailored to the unique characteristics of on-policy DRL training with repeated epochs.

III. PROBLEM STATEMENT

We consider a distributed RL setup where an agent learns to control the CarRacing-v3 environment through interaction. The agent consists of a convolutional neural network (CNN) for feature extraction and an actor-critic network for policy and value estimation. Given resource constraints, we partition this agent across two machines: Machine 0 hosts the CNN and environment interaction, while Machine 1 hosts the actor-critic network.

A. Communication Overhead Analysis

We compare data transfer requirements across two deployment scenarios over 100 training iterations with batch size 4096 (100 iterations $\times$ 4096 steps per iteration).

Cloud Setup: The local machine transmits raw observations to the cloud for complete training. Each observation is a $4 \times 96 \times 96$ tensor. Total data transfer:

$$100 \times 4096 \times (4 \times 96 \times 96) = 14.7\text{B tensors} \approx 28 \text{ GB} \quad (1)$$

Pipeline-Parallel Setup: The CNN runs locally, transmitting activations to Machine 1 and receiving gradients. With 10 update epochs per batch and 32 minibatches per epoch (minibatch size 128), we transmit 4096-dimensional activation and gradient vectors. Total data transfer:

$$100 \times 10 \times 32 \times 128 \times 4096 \times 2 = 32.7\text{B tensors} \approx 66 \text{ GB} \quad (2)$$

The factor of 2 accounts for bidirectional communication (forward activations and backward gradients). While cloud setup requires 28 GB, pipeline parallelism requires 66 GB—a $2.4\times$ increase. Table I summarizes this comparison.

TABLE I
COMMUNICATION OVERHEAD COMPARISON

Setup	Data Transferred	Relative Cost
Cloud Setup	28 GB	1.0 $\times$
Pipeline-Parallel	66 GB	2.4 $\times$

This overhead stems from repeated gradient exchanges during multiple training epochs. Our goal is to reduce backward gradient communication while maintaining training performance, making pipeline parallelism viable for bandwidth-constrained environments.

IV. METHOD

A. Experimental Setup

We train a reinforcement learning agent on the CarRacing-v3 environment from OpenAI Gym, where the goal is to control a car around a racing track by outputting continuous steering, acceleration, and braking actions. Each observation is a $4 \times 96 \times 96$ RGB image stack representing four consecutive frames. The agent employs Proximal Policy Optimization (PPO) with a total training budget of 500,000 timesteps across 100 iterations, where each iteration consists of 4,096 environment steps followed by 10 update epochs with 32 minibatches of size 128.

To simulate pipeline parallelism across bandwidth-constrained devices, we deploy our system using Docker Compose with two containers connected via a bridge network. Machine 0 hosts the environment and CNN feature extractor, while Machine 1 hosts the actor-critic network. Communication between machines uses PyTorch’s distributed RPC framework, which transmits tensors over the network and provides a realistic simulation of inter-device bandwidth limitations.

B. Network Architecture and Partitioning

The agent network consists of two components distributed across machines:

CNN Feature Extractor (Machine 0): Processes raw observations through three convolutional layers: Conv2d(4 $\rightarrow$ 32,

kernel=8, stride=4), Conv2d(32→64, kernel=4, stride=2), and Conv2d(64→64, kernel=3, stride=1), each followed by ReLU activation. The output is flattened to a 4,096-dimensional feature vector. This compression from $4 \times 96 \times 96 = 36,864$ input dimensions to 4,096 features provides an $9\times$ reduction, significantly lowering communication overhead during policy rollout compared to transmitting raw observations.

Actor-Critic Network (Machine 1): Takes the 4,096-dimensional features as input. The actor head produces a 3-dimensional continuous action (steering, acceleration, brake) through a two-layer MLP (4096→512→3) with Gaussian policy parameterization and tanh squashing. The critic head estimates state values through a parallel MLP (4096→512→1). Both networks use orthogonal weight initialization for stable training.

During policy rollout, Machine 0 sends 4,096-dimensional feature vectors to Machine 1 for action selection—a $9\times$ bandwidth savings compared to sending raw $4 \times 96 \times 96$ observations. However, the training phase requires bidirectional communication: forward activations and backward gradients must be exchanged for each minibatch across 10 epochs, creating the 66 GB overhead identified in Section III.

C. Gradient Accumulation with Percentile-Based Sparsification

To reduce backward gradient communication, we introduce a selective gradient transmission strategy inspired by Deep Gradient Compression [5]. Rather than transmitting full gradient tensors from Machine 1 to Machine 0 after each minibatch, we accumulate gradients locally on Machine 1 and transmit only the most significant values.

Accumulation Phase: Machine 1 maintains a global gradient accumulator $\mathbf{G} \in \mathbb{R}^{b \times d}$ initialized to zero, where b is the minibatch size (128) and d is the feature dimension (4,096). After computing the backward pass for minibatch i , we accumulate the feature gradients:

$$\mathbf{G} \leftarrow \mathbf{G} + \nabla_{\mathbf{f}_i} \mathcal{L} \quad (3)$$

where $\nabla_{\mathbf{f}_i} \mathcal{L}$ represents gradients with respect to CNN features for minibatch i .

Sparsification and Transmission: After accumulation, we compute the p -th percentile threshold on absolute gradient magnitudes:

$$\tau = \text{quantile}(|\mathbf{G}|, p) \quad (4)$$

where $p \in [0.90, 0.99]$ is a configurable hyperparameter. We create a binary mask $\mathbf{M}$ and transmit only gradients exceeding the threshold:

$$\mathbf{M}_{ij} = \begin{cases} 1 & \text{if } |\mathbf{G}_{ij}| \geq \tau \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

The sparse gradient representation consists of three components: (1) indices of non-zero elements stored as int16 coordinates, (2) corresponding gradient values in bfloat16, and (3) tensor shape metadata. For a 90th percentile threshold, we transmit only 10% of gradient values, achieving a theoretical

$10\times$ compression. After transmission, selected gradients are zeroed in the accumulator:

$$\mathbf{G}_{ij} \leftarrow \begin{cases} 0 & \text{if } \mathbf{M}_{ij} = 1 \\ \mathbf{G}_{ij} & \text{otherwise} \end{cases} \quad (6)$$

Reconstruction on Machine 0: Machine 0 reconstructs the sparse gradient tensor from received indices and values, then applies it to update CNN parameters through standard backpropagation with gradient clipping (max norm 0.5) and Adam optimization (learning rate 5×10^{-5}).

Buffer Decay: To mitigate potential staleness in the accumulator, we optionally apply an exponential decay factor $\lambda \in (0, 1]$ before adding new gradients:

$$\mathbf{G} \leftarrow \lambda \mathbf{G} + \nabla_{\mathbf{f}_i} \mathcal{L} \quad (7)$$

Setting $\lambda = 1.0$ recovers the standard accumulation. Values below 1.0 discount older, unsent gradients so that the accumulator reflects more recent optimization directions.

Warm Start: To ensure stable initial training, we apply full gradient communication for the first 30,000 timesteps before enabling gradient accumulation. This allows the network to establish reasonable initial weights before introducing compression.

The key advantage of this approach for RL is that repeated training epochs naturally generate multiple gradient signals for similar data, making accumulation particularly effective. By deferring transmission until gradients become statistically significant, we reduce communication while preserving the most informative updates for the feature extractor.

V. EVALUATION RESULTS

We evaluate our gradient accumulation approach across four configurations over 723,000 training steps on CarRacing-v3: (1) baseline pipeline parallelism without compression, (2) 90th percentile sparsification, (3) 90th percentile with 0.99 buffer decay, and (4) 90th percentile with 0.95 buffer decay. Episodic returns are smoothed using exponential moving average (EMA) with decay 0.9.

A. Training Performance

Figure 1 shows training performance. The vertical dashed line marks the warm start boundary at 30,000 steps. The baseline without compression continues improving throughout training, reaching an EMA return of 760 by 723,000 steps. The 90th percentile configuration without decay tracks the baseline initially but gradually diverges, reaching an EMA return of 288 at 723,000 steps. Both decay variants peak earlier—the 0.95 decay around 250,000 steps and the 0.99 decay around 350,000 steps—then degrade, ending at EMA returns of 85 and -17 respectively.

B. Communication Overhead

Figure 2 presents cumulative data transfer. The baseline transmits 242 GB over 723,000 steps. All three 90th percentile configurations—with and without decay—transfer approximately 157 GB, achieving a 35% reduction (85 GB saved).

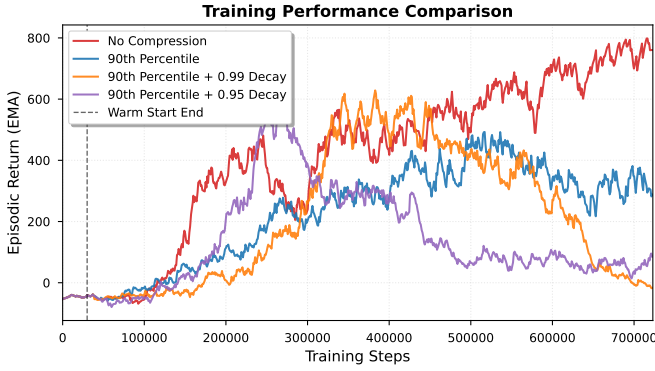


Fig. 1. Training performance comparison over 723,000 steps. The baseline continues improving throughout training. The 90th percentile without decay diverges gradually. Both decay variants ($\lambda = 0.99, 0.95$) peak early then degrade. The dashed vertical line indicates warm start completion at 30,000 steps.

The decay factor does not affect communication volume since the same percentile threshold determines what is transmitted; it only affects the magnitude of retained gradients.

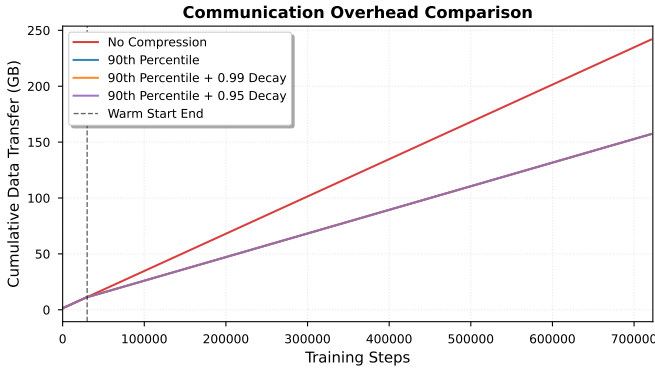


Fig. 2. Cumulative network transfer comparison. All 90th percentile configurations achieve identical 35% bandwidth reduction regardless of decay factor, saving 85 GB over 723,000 training steps.

C. Buffer Decay Analysis

The decay variants were motivated by a concern that below-threshold gradients persisting indefinitely in the accumulator may become stale as the policy evolves. We apply an exponential decay factor λ to the accumulator before adding new gradients each minibatch, discounting older unsent values.

However, the decay interacts poorly with the per-minibatch accumulation schedule. With 32 minibatches per epoch and 10 update epochs, the decay is applied 320 times per training iteration. At $\lambda = 0.95$, retained gradients are scaled by $0.95^{320} \approx 7.5 \times 10^{-8}$ after one iteration—effectively zeroing out any gradient that does not exceed the percentile threshold within a single iteration. At $\lambda = 0.99$, the retention factor is $0.99^{320} \approx 0.04$, which is less aggressive but still substantial. In both cases, the decay prevents below-threshold gradients from accumulating to significance, reducing the gradient signal

available to the CNN feature extractor and causing performance to degrade after an initial peak.

These results indicate that multiplicative buffer decay is counterproductive when applied at the minibatch granularity. The percentile-based sparsification already manages the accumulator by zeroing out transmitted values; adding decay on top removes gradient signal before it can accumulate to the transmission threshold.

D. Policy Evaluation

To further characterize final policy quality, we evaluated trained checkpoints over 100 independent environment episodes with a fixed random seed. Each checkpoint was loaded after training completed and rolled out in CarRacing-v3 without any further updates. Figure 3 shows box plots of episodic returns for the baseline and 90th percentile configurations. The baseline achieves a mean return of 398.97 (± 226.58) with a median of 376.68, while gradient accumulation achieves a mean of 305.32 (± 161.73) with a median of 283.18. The interquartile ranges substantially overlap—the gradient accumulation Q1–Q3 range of [190, 416] versus the baseline range of [284, 520]—and the 90th percentile model achieves 76.5% of baseline mean performance. The tighter standard deviation under compression (161.73 vs. 226.58) suggests the compressed model produces more consistent, if somewhat lower, returns.

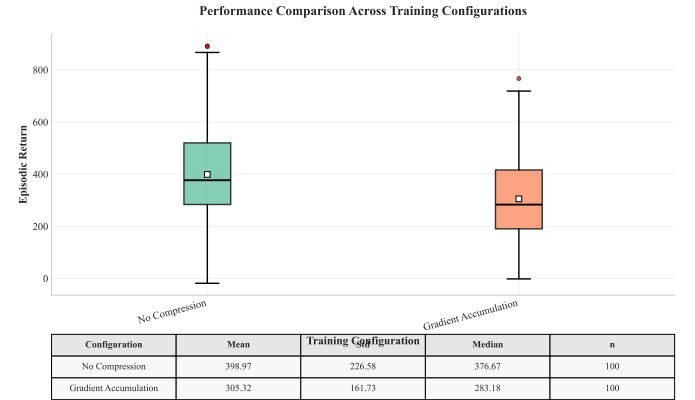


Fig. 3. Evaluation return distributions over 100 independent episodes per checkpoint (no further training). Box plots show median, IQR, and whiskers ($1.5 \times \text{IQR}$). Gradient accumulation (90th percentile) achieves 76.5% of baseline mean return with overlapping IQRs.

REFERENCES

- [1] A. Nair, P. Srinivasan, S. Blackwell, C. Alcicek, R. Fearon, A. De Maria, V. Panneershelvam, M. Suleyman, C. Beattie, S. Petersen, et al., “Massively parallel methods for deep reinforcement learning,” in *ICML Deep Learning Workshop*, 2015.
- [2] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, et al., “IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures,” in *Proc. ICML*, 2018, pp. 1407–1416.
- [3] D. Horgan, J. Quan, D. Budden, G. Barth-Maron, M. Hessel, H. Van Hasselt, and D. Silver, “Distributed prioritized experience replay,” in *Proc. ICLR*, 2018.

- [4] W. Chen, S. Wang, Y. Hong, and G. B. Giannakis, "Communication-efficient policy gradient methods for distributed reinforcement learning," *IEEE Trans. Signal Process.*, vol. 69, pp. 2629–2641, 2021.
- [5] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," in *Proc. ICLR*, 2018.
- [6] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. NeurIPS*, 2017, pp. 1709–1720.
- [7] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Proc. NeurIPS*, 2019, pp. 103–112.
- [8] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, "PipeDream: Generalized pipeline parallelism for DNN training," in *Proc. ACM SOSP*, 2019, pp. 1–15.
- [9] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," arXiv preprint arXiv:1909.08053, 2019.
- [10] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," in *Proc. IEEE SC*, 2020, pp. 1–16.
- [11] H. Lim, Y. J. Kim, and M. L. Sichitiu, "Federated reinforcement learning for training control policies on multiple IoT devices," *Sensors*, vol. 20, no. 5, p. 1359, 2020.
- [12] X. Feng, C. Luo, J. Chen, Y. Huang, J. Zhang, W. Xu, J. Li, and V. C. M. Leung, "IoTSL: Toward efficient distributed learning for resource-constrained Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 11, pp. 9892–9905, 2023.